📐 MATHEMATICS FOR MACHINE LEARNING

CampusX · Data Science Mentorship Program 2023 · Video 6 of 23

NON-GAUSSIAN PROBABILITY DISTRIBUTIONS

Session 42 — Uniform, Log-Normal, and Pareto Distributions · Kurtosis · Data Transformations

Kurtosis Deep Dive | Uniform Distribution | Log-Normal Distribution | Pareto Distribution | QQ Plots Analysis | Mathematical Transforms | Box-Cox & Yeo-Johnson

---

Session Notes — Complete Reference

Playlist: Maths for Machine Learning · Instructor: Nitish Singh (CampusX)

📑 Table of Contents

01 Introduction to Non-Gaussian Distributions

02 Kurtosis & Excess Kurtosis

03 Continuous Uniform Distribution

04 Log-Normal Distribution

05 Pareto Distribution (Power Law)

06 Evaluating Distributions with QQ Plots

07 Why Do We Transform Data?

08 Mathematical Data Transformations

09 Power Transformers — Box-Cox & Yeo-Johnson

10 Sklearn API Integration

11 Python Implementation — Code Examples

12 Quick Reference Card

1 · Introduction to Non-Gaussian Distributions

While the Normal distribution is very common, real-world datasets often do not follow a Gaussian curve. Many domains exhibit patterns that are bounded, highly skewed, or fat-tailed. Modeling these correctly is essential for building accurate statistical models and machine learning pipelines.

1.1 What are Non-Gaussian Distributions?

A non-Gaussian distribution is any probability distribution that does not follow the standard bell-shaped symmetric Gaussian curve. Common characteristics include:

- **Asymmetric shape:** Skewed to the left or right
- **Bounded domains:** Variables that cannot be negative or must fall in a fixed range
- **Heavy tails:** Outliers occur far more frequently than predicted by a normal curve
- **Flat structures:** Outcomes having equal probabilities over an interval

> 💡 **Why Do We Study Them?**
>
> Using algorithms that assume Gaussian data (like Linear Regression or Logistic Regression) on highly non-Gaussian features can lead to biased parameters, poor predictions, and incorrect hypothesis test conclusions. Recognizing the type of distribution allows us to choose either non-parametric methods, algorithms that handle arbitrary distributions (like tree-based models), or apply data transformations to make the distribution Gaussian-like.

2 · Kurtosis & Excess Kurtosis

Kurtosis is a statistical measure that defines how heavily the tails of a distribution differ from the tails of a normal distribution. It describes the extreme values (outliers) present in the data.

2.1 The Mathematical Formula

KURTOSIS FORMULA (FOURTH STANDARDIZED MOMENT)

$$\text{Kurtosis } (\beta_2) = E[(X - \mu)^4] / \sigma^4 \quad \text{Excess Kurtosis} = \text{Kurtosis} - 3$$

2.2 Three Types of Kurtosis

📊 Mesokurtic (Kurtosis = 3)

This is the baseline behavior of the **Normal Distribution**. Its excess kurtosis is exactly **0**.

Outliers occur at standard expected rates.

📊 Leptokurtic (Kurtosis > 3)

Excess kurtosis is positive (**> 0**). The distribution has **heavy tails** and a sharp, tall peak.

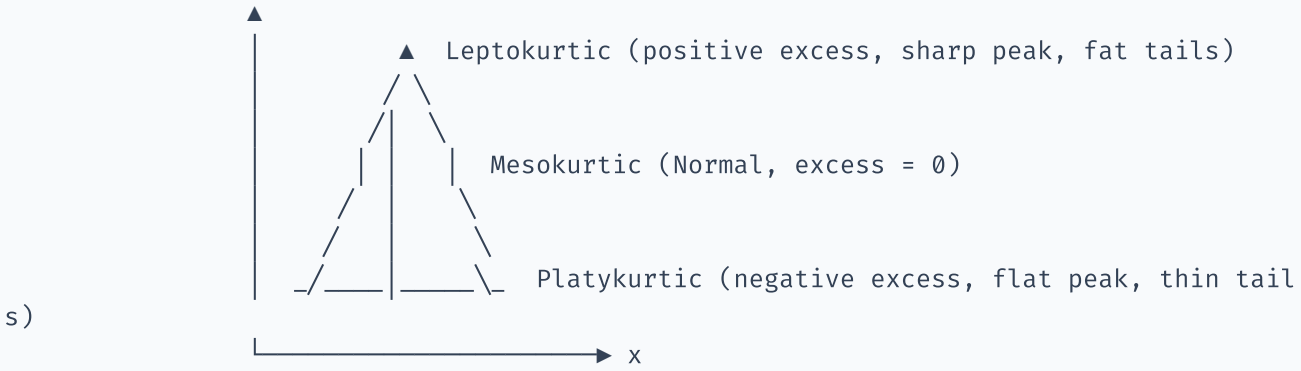
Think: High risk of extreme outliers. Examples: Laplace and Student's t-distributions.

📊 Platykurtic (Kurtosis < 3)

Excess kurtosis is negative (**< 0**). The distribution has **thin tails** and a flatter, shorter peak.

Think: Low risk of outliers. Example: Uniform distribution.

📊 Visualizing Kurtosis Types



3 · Continuous Uniform Distribution

The **Continuous Uniform Distribution** describes an experiment where every outcome within a bounded interval $[a, b]$ is equally likely to occur.

3.1 PDF and CDF Formulas

UNIFORM DISTRIBUTION FORMULAS

```
PDF: f(x) = 1 / (b - a) for a ≤ x ≤ b, and 0 elsewhere CDF: F(x) = (x - a) / (b - a) for a ≤ x ≤ b
```

3.2 Properties

PROPERTY	FORMULA / VALUE
Notation	$X \sim U(a, b)$
Mean (Expected Value)	$E[X] = (a + b) / 2$
Variance	$Var(X) = (b - a)^2 / 12$
Standard Deviation	$\sigma = (b - a) / \sqrt{12}$
Skewness	0 (perfectly symmetric)
Kurtosis	1.8 (excess kurtosis = –1.2 → platykurtic)

📝 **Real-World Use Case**

- **Random number generation:** Generating a random floating-point number between 0 and 1 (np.random.rand()).
- **Uniform priors:** Used in Bayesian statistics when we want to express complete uncertainty about a parameter's range.

4 · Log-Normal Distribution

A continuous random variable X is **log-normally distributed** if its natural logarithm $Y = \ln(X)$ follows a Normal distribution.

4.1 PDF and CDF

LOG-NORMAL PDF

`f(x) = (1 / (x·σ·√(2π))) · e^[-(ln(x) - μ)² / (2σ²)] for x > 0 where: μ and σ are the mean and standard deviation of the variable's ln(X) (NOT the mean and standard deviation of X itself).`

4.2 Properties

- **Domain:** $(0, +\infty)$. A log-normal variable can never take negative values or zero.
- **Shape:** Right-skewed (positively skewed) distribution.
- **Mean of X:** $e^{\mu + \sigma^2/2}$
- **Median of X:** e^{μ}
- **Mode of X:** $e^{\mu - \sigma^2}$
- **Relationship:** As $\sigma \to 0$, the log-normal distribution approaches a normal shape.

📝 **Real-World Examples**

• **Income and wealth distribution:** Most people earn near the median, but a small fraction earns exponentially more.
• **Biometric properties:** Rainfall amounts, incubation periods of diseases, sizes of living organisms.
• **Finance:** Stock prices are often modeled as log-normal because prices cannot drop below zero.

5 · Pareto Distribution (Power Law)

The **Pareto Distribution** is a power-law probability distribution that is widely used to describe social, scientific, and geophysical phenomena. It is highly skewed and features an extremely heavy tail, embodying the famous "80/20 rule".

5.1 Formulas

PARETO DISTRIBUTION FORMULAS

```
PDF: f(x) = (α · x_m^α) / x^(α + 1) for x ≥ x_m CDF: F(x) = 1 - (x_m / x)^α for
x ≥ x_m where: x_m = scale parameter (minimum possible value of X, must be > 0)
α = shape parameter (tail index, determines thickness of tail)
```

5.2 Properties

PROPERTY	CONDITION / VALUE
Mean	$E[X] = (\alpha \cdot x_m) / (\alpha - 1)$ (only defined if $\alpha > 1$)
Variance	$Var(X) = (\alpha \cdot x_m^2) / ((\alpha - 1)^2(\alpha - 2))$ (only defined if $\alpha > 2$)
80/20 Rule	Approximately 80% of effects come from 20% of causes (occurs when $\alpha \approx 1.16$)

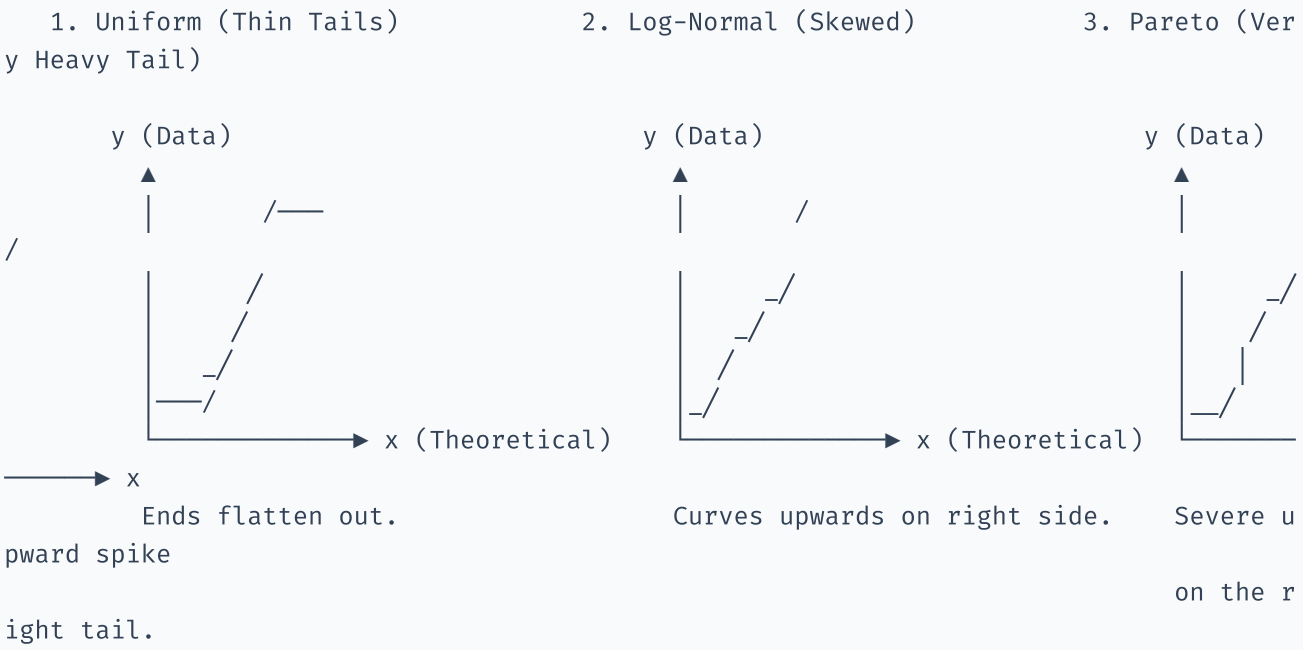
📝 **Real-World Examples**

- **Wealth distribution:** 80% of the world's wealth is owned by 20% of the population.
- **City sizes:** A few cities have massive populations, while most have very small populations.
- **Software engineering:** 80% of execution time is spent running 20% of the code.

6 · Evaluating Distributions with QQ Plots

The **Quantile–Quantile (Q–Q) Plot** is an essential visual diagnostic tool to check if empirical data matches a theoretical distribution (usually Normal, but can be set to any distribution).

📊 Q-Q Plot Deviation Signatures (compared to Normal)



7 · Why Do We Transform Data?

Many classic machine learning models assume that numerical features follow a Normal (Gaussian) distribution. Transforming data is the process of modifying the scale of our features to satisfy this assumption.

7.1 Model Assumptions

- **Linear Models:** Linear and Logistic Regression assume that the residuals (errors) are normally distributed.
- **Parametric Models:** Linear Discriminant Analysis (LDA) and Gaussian Naive Bayes assume normal distributions of the features.
- **Distance-based models:** Highly skewed features can distort distance metrics (e.g., KNN, K-Means).

7.2 Benefits of Normality Transformations

1. **Stabilizes variance:** Makes variance homoscedastic (constant across the range of predictions)
2. **Reduces the influence of outliers:** Expands dense regions and compresses sparse, long-tailed regions
3. **Linearizes relationships:** Makes non-linear relationships more linear
4. **Improves model convergence:** Gradient descent algorithms converge much faster on normally scaled features

8 · Mathematical Data Transformations

TRANSFORMATION	FORMULA	USE CASE	LIMITATIONS
Log Transform	$X' = \ln(X)$ or $\ln(X + 1)$	Highly right-skewed data, log-normal data	Cannot handle zero or negative values (use ln(X+1) for zero)
Reciprocal Transform	$X' = 1 / X$	Inverts large and small values (e.g., speed to travel time)	Cannot handle zero values
Square Root	$X' = \sqrt{X}$	Moderate right-skewed data, count data (Poisson)	Cannot handle negative values
Square Transform	$X' = X^2$	Left-skewed data (left tail compression)	Usually applied to negative/bounded ranges

⚠️ Transform Target vs Features

In regression, we often transform the **target variable (y)** to stabilize residual variance. If you transform the target, remember to apply the **inverse transform** (e.g., $e^{y'}$ for log transform) to your predictions to return them to the original scale.

9 · Power Transformers — Box-Cox & Yeo-Johnson

Rather than manually guessing which mathematical function to apply, we can use **Power Transformers** which automatically find the optimal mathematical parameter (λ) to make the data as Gaussian-like as possible.

9.1 Box-Cox Transformation

BOX-COX TRANSFORMATION FORMULA

$$y_i^{(\lambda)} = \begin{cases} \frac{x_i^{\lambda} - 1}{\lambda} & \text{if } \lambda \neq 0 \\ \ln(x_i) & \text{if } \lambda = 0 \end{cases}$$

Constraint: x_i must be strictly greater than 0 ($x_i > 0$).

9.2 Yeo-Johnson Transformation

The Yeo-Johnson transformation is an extension of the Box-Cox transformation that allows for **zero and negative values**.

YEO-JOHNSON TRANSFORMATION FORMULA

$$y_i^{(\lambda)} = \begin{cases} \frac{(x_i + 1)^{\lambda} - 1}{\lambda} & \text{if } \lambda \neq 0,\ x_i \geq 0 \\ \ln(x_i + 1) & \text{if } \lambda = 0,\ x_i \geq 0 \\ -\frac{(-x_i + 1)^{(2-\lambda)} - 1}{(2-\lambda)} & \text{if } \lambda \neq 2,\ x_i < 0 \\ -\ln(-x_i + 1) & \text{if } \lambda = 2,\ x_i < 0 \end{cases}$$

💡 **Finding the Optimal Lambda**

Algorithms like Maximum Likelihood Estimation (MLE) or Pearson correlation optimization are used in background libraries to scan potential λ values and pick the one that maximizes the normality of the output.

10 · Sklearn API Integration

Scikit-Learn provides dedicated classes to apply these transformations in machine learning pipelines.

1. FunctionTransformer

Used to apply custom mathematical functions (like log, sqrt, reciprocal) within pipelines.

2. PowerTransformer

Used to automatically apply Box-Cox or Yeo-Johnson transformations to features.

> 💡 **Pipeline Safety**
>
> Using `PowerTransformer` from Scikit-Learn is essential because it learns the optimal parameters (λ, mean, scale) on the **training set** and applies those same parameters to the **test set**, preventing data leakage.

11 · Python Implementation — Code Examples

11.1 Plotting Non-Gaussian Distributions

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
import seaborn as sns

# Generate Uniform, Log-Normal, and Pareto distributions
np.random.seed(42)
uniform_data = np.random.uniform(low=0, high=10, size=1000)
lognormal_data = np.random.lognormal(mean=1, sigma=0.6, size=1000)
pareto_data = (np.random.pareto(a=2, size=1000) + 1) * 2  # scale=2, alpha=2

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
sns.histplot(uniform_data, kde=True, ax=axes[0], color='#3b82f6')
axes[0].set_title('Uniform Distribution')

sns.histplot(lognormal_data, kde=True, ax=axes[1], color='#7c3aed')
axes[1].set_title('Log-Normal Distribution')

sns.histplot(pareto_data, kde=True, ax=axes[2], color='#059669')
axes[2].set_xlim(0, 20)
axes[2].set_title('Pareto Distribution (Power Law)')
plt.tight_layout()
plt.show()
```

11.2 Calculating Kurtosis

```
# Calculate excess kurtosis (Fisher's definition, where Normal = 0)
k_uni = stats.kurtosis(uniform_data, fisher=True)
k_log = stats.kurtosis(lognormal_data, fisher=True)
k_par = stats.kurtosis(pareto_data, fisher=True)

print(f"Uniform excess kurtosis: {k_uni:.4f} (Platykurtic)")
print(f"Log-Normal excess kurtosis: {k_log:.4f} (Leptokurtic)")
print(f"Pareto excess kurtosis: {k_par:.4f} (Highly Leptokurtic)")
```

11.3 FunctionTransformer in Pipelines

```
from sklearn.preprocessing import FunctionTransformer
from sklearn.pipeline import Pipeline
import pandas as pd

df = pd.DataFrame({'salary': [50000, 80000, 120000, 300000, 1000000]})

# Apply Log Transform using Sklearn FunctionTransformer
log_transformer = FunctionTransformer(np.log1p)  # log(x + 1)
df_transformed = log_transformer.transform(df)
print(df_transformed)
```

11.4 PowerTransformer (Box-Cox & Yeo-Johnson)

```
from sklearn.preprocessing import PowerTransformer

# Box-Cox requires values > 0
pt_box = PowerTransformer(method='box-cox')
boxcox_salary = pt_box.fit_transform(df[['salary']])
print(f"Box-Cox Lambda: {pt_box.lambdas_[0]:.4f}")

# Yeo-Johnson can handle negative/zero values
pt_yeo = PowerTransformer(method='yeo-johnson')
yeo_salary = pt_yeo.fit_transform(df[['salary']])
print(f"Yeo-Johnson Lambda: {pt_yeo.lambdas_[0]:.4f}")
```

12 · Quick Reference Card

🚀 Session 42 — Non-Gaussian Distributions Cheat Sheet

Kurtosis Summary

- **Mesokurtic:** Kurtosis = 3 (Excess = 0) → Normal behavior
- **Leptokurtic:** Kurtosis > 3 (Excess > 0) → Heavy tails, high outlier risk
- **Platykurtic:** Kurtosis < 3 (Excess < 0) → Flat shape, thin tails

Key Distributions

- **Uniform:** PDF = 1/(b−a), Flat probability density
- **Log-Normal:** Bounded [0, ∞), Right-skewed. ln(X) follows Normal
- **Pareto:** Heavy-tailed power law. 80/20 rule

Data Transformations

- **Log:** Compresses right-skewed data. Handles exponential relationships
- **Square Root:** Good for moderately right-skewed and count data
- **Reciprocal:** 1/x transform, flips scales
- **Box-Cox:** Power transform. Automatically finds best λ. Requires data > 0
- **Yeo-Johnson:** Power transform. Works on positive, zero, and negative data

Python Sklearn API

- `FunctionTransformer(np.log1p)` — Custom custom transformations
- `PowerTransformer(method='box-cox')` — Automatic power transformation
- `PowerTransformer(method='yeo-johnson')` — Zero/negative value friendly

📚 What's Next? → Central Limit Theorem (Session 43)

The next session covers the **Central Limit Theorem (CLT)**, explaining how sample means converge to a normal distribution regardless of the underlying population distribution, forming the core of inferential statistics.